

AIops A2

Krish Dange

September 2026

Q1. Docker Image Optimization

0.1 1. Single-Stage Dockerfile

A single-stage Dockerfile was created using the `python:3.11-slim` base image. The resulting Docker image was successfully built and the measured image size was:

Single-stage image size: 421 MB

0.2 2. Multi-Stage Dockerfile

The Dockerfile was rewritten using a multi-stage build. The builder stage uses `python:3.11` to install the required dependencies, while the final stage uses `python:3.11-slim`.

The measured image size was:

Multi-stage image size: 411 MB

The percentage reduction is:

$$\frac{421 - 411}{421} \times 100 = \mathbf{2.38\%}$$

0.3 3. Explanation

The builder stage uses the full `python:3.11` image to install the required Python dependencies, while the final image uses the smaller `python:3.11-slim` image. The full builder environment and its additional files are left behind, so only the installed dependencies and application files are copied into the final image.

1 Q2. Docker Compose and Redis Caching

1.1 1. Redis Caching

The API was modified to use Redis as a cache for prediction results. For each request, the API first checks Redis using the input text as the key. On a cache

miss, the model performs the prediction and the result is stored in Redis with a TTL of 300 seconds. On a cache hit, the stored label is returned directly without running the model again.

```

krish27@debian:~/aiops/AIops_A2$ docker compose logs api
api-1 | INFO: Started server process [1]
api-1 | INFO: Waiting for application startup.
api-1 | INFO: Application startup complete.
api-1 | INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
api-1 | INFO: 172.18.0.1:33862 - "GET /healthz HTTP/1.1" 200 OK
api-1 | INFO: 172.18.0.1:38816 - "GET /healthz HTTP/1.1" 200 OK
api-1 | INFO: 172.18.0.1:39250 - "GET /healthz HTTP/1.1" 200 OK
api-1 | Cache miss
api-1 | INFO: 172.18.0.1:47294 - "POST /predict HTTP/1.1" 200 OK
api-1 | Cache hit
api-1 | INFO: 172.18.0.1:34352 - "POST /predict HTTP/1.1" 200 OK
krish27@debian:~/aiops/AIops_A2$

```

Figure 1: Redis cache miss followed by cache hit

1.2 2. Docker Compose Configuration

A Docker Compose configuration was created with two services: the spam detection API and Redis. The API is built using the multi-stage Dockerfile, while Redis uses the official `redis:7-alpine` image. The API connects to Redis using the Compose service name `cache`.

```

krish27@debian:~/aiops/AIops_A2$ docker compose up -d
[+] Running 2/2
 ✓ Container aiops_a2-cache-1 Running 0.0s
 ✓ Container aiops_a2-api-1 Started 0.4s
krish27@debian:~/aiops/AIops_A2$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
aiops_a2-api-1	spam-api:multi	"uvicorn app.main:app..."	api	2 minutes ago	Up 10 seconds	0.0.0.0:8080->8080/tcp
aiops_a2-cache-1	redis:7-alpine	"docker-entrypoint.s..."	cache	2 minutes ago	Up 2 minutes	6379/tcp

```

krish27@debian:~/aiops/AIops_A2$

```

Figure 2: API and Redis services running with Docker Compose

1.3 3. Cache Performance

Two identical prediction requests were sent to the API. The first request resulted in a cache miss and took 0.071 seconds, while the second request was served from the cache and took 0.019 seconds.

```
iris@270debb1:~/alops/AlOps_A1$ docker compose up -d
[*] Running 2/2
✓ Container alops_a2-cache-1 Running 0.8s
✓ Container alops_a2-api-1 Started 0.4s
iris@270debb1:~/alops/AlOps_A1$ docker compose ps
NAME                IMAGE              COMMAND              SERVICE    CREATED      STATUS      PORTS
alops_a2-api-1      span-api:multi    "uvicorn app.main:ap..." api        2 minutes ago Up 10 seconds 0.0.0.0:8080->8080/tcp
alops_a2-cache-1    redis:7-alpine     "docker-entrypoint.s..." cache      2 minutes ago Up 2 minutes 6379/tcp
```

Figure 3: Comparison of first and second identical requests

The second request was approximately 3.7 times faster in this measurement, demonstrating the benefit of caching repeated requests.

1.4 4. Docker Compose vs. Kubernetes

Docker Compose is useful for running multiple related containers together on a single host, such as the API and Redis in this application. Kubernetes provides cluster-level container orchestration and is designed for larger distributed deployments, including features such as scheduling, self-healing, scaling, and rolling updates.

2 Q3. Kubernetes Indexed Job

2.1 1. Indexed Job Configuration

An Indexed Kubernetes Job was created to process 8 partitioned CSV shards containing synthetic user signup records. Each shard contains 20 records, including malformed email addresses and missing required fields.

The Job was configured with 8 completions and a parallelism of 4 using Indexed completion mode:

```
completions: 8
```

parallelism: 4
completionMode: Indexed

Each pod receives a unique completion index from 0 to 7 and uses it to process exactly one corresponding CSV shard. The pod name and node name were provided to the container using the Kubernetes Downward API.

Each pod was configured with a CPU request and limit of 500m. With two nodes and an assumed 2 allocatable CPUs per node, four pods can run concurrently, so a parallelism of 4 was selected.

2.2 2. Job Execution and Validation

The configured parallelism was successfully reached. Four validation pods were observed running concurrently across the two Minikube nodes.

```

crish27@debian:~/aiops/AIops_A3/a3$ kubectl apply -f a3-job.yaml
job.batch/signup-validation-job created
crish27@debian:~/aiops/AIops_A3/a3$ kubectl get pods -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GA
tes								
hp-search-job-8-pgrs8	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-1-v4jtj	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-10-b5vn9	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-11-mwdr	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-2-qftjp	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-3-9f624	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-4-slij2	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-5-sgkq9	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-6-7sejn	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-7-hsjjn	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-8-wpcr8	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
hp-search-job-9-p4t28	0/1	Completed	0	12d	<none>	minikube	<none>	<none>
signup-validation-job-0-skdw6	1/1	Running	0	10s	10.244.1.2	minikube-m02	<none>	<none>
signup-validation-job-1-2z4bf	1/1	Running	0	10s	10.244.1.3	minikube-m02	<none>	<none>
signup-validation-job-2-4onnd	1/1	Running	0	10s	10.244.1.4	minikube-m02	<none>	<none>
signup-validation-job-3-mgmxx	1/1	Running	0	10s	10.244.0.3	minikube	<none>	<none>

Figure 4: Four Indexed Job pods running concurrently across the two nodes.

The Job completed successfully with all eight completions:

```
signup-validation-job Complete 8/8 29s
```

```

krish2@minikube:~/aiops/AIops_A2/q3$ kubectl get pods --o wide
NAME                                READY    STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
tes
hp-search-job-0-pgmr9               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-1-v4itj               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-10-05vrb              0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-11-nwudr              0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-2-qftjp               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-3-9FE24               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-4-slj2                0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-5-9geu9               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-6-7snjn               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-7-hckjm               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-8-spcr8               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
hp-search-job-9-p4t28               0/1      Completed 0           12d   <none>          minikube         <none>            <none>
signup-validation-job-0-8kub        1/1      Running   0           10s   10.244.1.2      minikube-m02    <none>            <none>
signup-validation-job-1-2z4bf       1/1      Running   0           10s   10.244.1.3      minikube-m02    <none>            <none>
signup-validation-job-2-fvwd        1/1      Running   0           10s   10.244.1.4      minikube-m02    <none>            <none>
signup-validation-job-3-mprnx       1/1      Running   0           10s   10.244.0.3      minikube         <none>            <none>
krish2@minikube:~/aiops/AIops_A2/q3$ kubectl get jobs
NAME                                STATUS    COMPLETION   DURATION   AGE
hp-search-job                       Complete  12/12        34s        12d
signup-validation-job               Complete  0/0          29s        119s

```

Figure 5: Successful completion of all eight Job shards.

Each shard contained 20 records and produced two invalid rows. The completed pod logs were retrieved using the Kubernetes Python client through the Kubernetes API.

Shard	Node	Invalid Rows
0	minikube-m02	2
1	minikube-m02	2
2	minikube-m02	2
3	minikube	2
4	minikube	2
5	minikube-m02	2
6	minikube-m02	2
7	minikube-m02	2

```

[qi3-venv] krish27@debian:~/staps/ADops_A2/q1$ python read_job_logs.py
Shard 0: Pod=signup-validation-job-9-s4d6b, Node=minikube-m02, Invalid rows=2
Shard 1: Pod=signup-validation-job-1-224bf, Node=minikube-m02, Invalid rows=2
Shard 2: Pod=signup-validation-job-2-fuend, Node=minikube-m02, Invalid rows=2
Shard 3: Pod=signup-validation-job-3-nqms, Node=minikube, Invalid rows=2
Shard 4: Pod=signup-validation-job-4-6cjcs, Node=minikube, Invalid rows=2
Shard 5: Pod=signup-validation-job-5-przcr, Node=minikube-m02, Invalid rows=2
Shard 6: Pod=signup-validation-job-6-srs78, Node=minikube-m02, Invalid rows=2
Shard 7: Pod=signup-validation-job-7-jdb7h, Node=minikube-m02, Invalid rows=2
[qi3-venv] krish27@debian:~/staps/ADops_A2/q1$

```

Figure 6: Pod logs retrieved through the Kubernetes API showing shard, node, and invalid-row information.

2.3 3. Storage and Scaling Considerations

Minikube’s storage provisioner provides dynamic persistent-volume provisioning when persistent storage is required. For this workload, a shared volume was not necessary because the CSV shards were packaged into the validator image and the results were collected through the Kubernetes API.

With 3 nodes and 2 allocatable CPUs per node, the available capacity would increase to 6 CPUs. With the same 500m CPU request per pod, **parallelism** could be increased from 4 to 6 while keeping **completions: 8**.

3 Q4. Kubernetes Deployment and Rolling Update

3.1 1. Deployment and Service

The spam detection API was deployed using a Kubernetes Deployment with two replicas. Each pod was configured with CPU and memory requests and limits, along with a readiness probe on the **/healthz** endpoint. A ClusterIP Service named **spam-api** was also created to provide stable access to the replicas on port 8080.

```

krish27@debian:~/aiops/AIops_A2/q1$ kubectl get pods -l app=span-api -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
span-api-3cf49fd745-efr8t	1/1	Running	0	8s	10.244.0.5	minikube	<none>	<none>
span-api-3cf49fd745-tf7bw	1/1	Running	0	8s	10.244.1.0	minikube-m02	<none>	<none>

```

krish27@debian:~/aiops/AIops_A2/q1$

```

Figure 7: Two API replicas running across the Minikube nodes.

The initial deployment returned the following response from the health endpoint:

```
{"status": "ok", "version": "v1"}
```

3.2 2. Self-Healing

To test Kubernetes self-healing, one running API pod was manually deleted. The Deployment controller detected the missing replica and automatically created a replacement pod. Both replicas returned to the **Running** state.

```

r1sh27gdeblan:~/aiops/aiops_A2/q1$ kubectl get pods -l app=spam-api -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
spam-api-5cf49fd745-nfr8t           1/1     Running   0           6m46s  10.244.0.5      minikube           <none>            <none>
spam-api-5cf49fd745-tf7hw           1/1     Running   0           6m46s  10.244.1.8      minikube-m02       <none>            <none>
r1sh27gdeblan:~/aiops/aiops_A2/q1$ kubectl delete pod spam-api-5cf49fd745-nfr8t
pod "spam-api-5cf49fd745-nfr8t" deleted from default namespace
r1sh27gdeblan:~/aiops/aiops_A2/q1$ kubectl get pods -l app=spam-api -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
spam-api-5cf49fd745-7fp4k           1/1     Running   0           7s    10.244.0.6      minikube           <none>            <none>
spam-api-5cf49fd745-tf7hw           1/1     Running   0           7m17s  10.244.1.8      minikube-m02       <none>            <none>
r1sh27gdeblan:~/aiops/aiops_A2/q1$

```

Figure 8: A replacement pod created after deleting a running replica.

This demonstrates the role of the Deployment and its ReplicaSet in maintaining the desired replica count.

3.3 3. Rolling Update

For the rolling update, the health endpoint was changed from **v1** to **v2**. A new image, **spam-api:k8s-v2**, was built and loaded into Minikube. The Deployment was then updated to use the new image.

The rollout completed successfully:

deployment "spam-api" successfully rolled out


```

krish27@debian:~/aiops/AIops_A3$ docker images spam-api
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
spam-api      k8s-v2    7cd8e3b4bd8b   9 seconds ago  417MB
spam-api      k8s-v1    7e010901cc87   13 minutes ago  417MB
spam-api      multi     8cac67fe8829   3 hours ago    417MB
spam-api      single    b6e2bf4e47b    5 hours ago    421MB
krish27@debian:~/aiops/AIops_A3$ minikube image load spam-api:k8s-v2
krish27@debian:~/aiops/AIops_A3$ minikube image ls | grep spam-api
docker.io/library/spam-api:k8s-v2
krish27@debian:~/aiops/AIops_A3$ kubectl set image deployment/span-api spam-api=spam-api:k8s-v2
deployment.apps/span-api image updated
krish27@debian:~/aiops/AIops_A3$ kubectl rollout status deployment/span-api
Waiting for deployment "span-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "span-api" rollout to finish: 1 old replicas are pending termination...
Deployment "span-api" successfully rolled out
krish27@debian:~/aiops/AIops_A3$

```

Figure 9: Successful rolling update of the spam API Deployment.

The rollout history contained two revisions, corresponding to the original deployment and the updated image.

Finally, the service was checked again through the health endpoint:

```
{"status": "ok", "version": "v2"}
```



```

krish27@debian:~$ curl http://127.0.0.1:8080/healthz
{"status": "ok", "version": "v2"}

```

Figure 10: Health endpoint returning the updated v2 application.

3.4 4. Deployment vs. Job

A Deployment is intended for long-running applications. It maintains the desired number of replicas, provides self-healing through its ReplicaSet, and sup-

ports rolling updates. A Job is intended for a finite batch workload and completes after the required number of successful executions, as demonstrated by the eight-shard validation workload in Q3.